

Figure 1: Differences in C++ and Java compilation

storage section, the instruction storage section and memory slots to perform operations. Each memory slot inside the stack frame is, by default, of 4 bytes, but according to the size of the variable, the size of the slot also shrinks or expands.

According to Figure 4, the addition of a 2-byte variable (a and b) will not result in a byte because the default size of the stack frame memory slot is 4 bytes, which can't be inserted into the byte (r) variable; so we need to typecast it as $(r = (\text{byte})a + b)$.

The PC register: The program counter (PC) register contains the address of the Java virtual machine instruction currently being executed.

Native method stacks: All the native methods are executed in this area.

Execution engine: All executions happening in JVM are controlled by the execution engine.

Native method interface: Java Native Interface (JNI) enables the Java code running in JVM to call and be called by the native application and libraries (Native Method Libraries) written in other languages such as C and C++.

JVM in action

Now let us take a look at a few Java code snippets and understand the role of various JVM components, during execution.

Example 1 (when all members are static): To understand this example, the method area is explained earlier in this article. According to the method area, all static members of a class are identified and executed in the same order in which they appear. When all static members are executed, only then is the main function executed.

```
class Test{
    static int a =m1();
    static{
        System.out.println ("in static block");
    }
    public static int m1(){
        System.out.println("in m1");
        return 10;
    }
    public static void main(String[] args){
        System.out.println("in main");
    }
}
```



Figure 2: How JVM fits between the OS and Java application

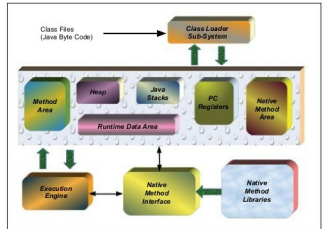


Figure 3: Architecture of JVM

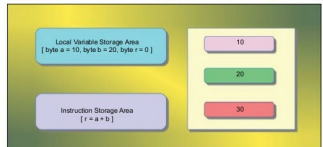


Figure 4: Architecture of the stack frame

```
}
The output of the above program is:
in m1
in static block
in main
```

Example 2 (in case of inheritance): To understand this example, you need to understand the method area as well as Example 1. In this example, we are trying to explain that, when a class is inherited from any other class, the static members are identified and executed from the top to the bottom or from parent class to child class, and the main function will be executed at the end.

```
class Vehicle{
    static int a = m1();
    public static int m1(){
```